# Developer onboarding

Welcome. This is everything **you** (a developer joining a project) need to do to start working in mykan and driving it from Claude Code. It picks up **after the project owner has added you** — two things happen on their side first:

1. **Your email is whitelisted** so you can sign in at all.
2. **The project is shared with you** so it's visible once you're in.

You don't do either of those — if you can't sign in or can't see the project, that's on the owner, not you (see Troubleshooting). Everything below is yours to do.

Use the **exact Google account** whose email the owner whitelisted. A different Google account — even yours — will be rejected. (For Gmail addresses, dots and `+tags` don't matter: `jane.doe@gmail.com` and `janedoe+work@gmail.com` are the same account, just as Google treats them.)

---

## 1. Sign in

1. Go to **https://kanban.dbwoodward.com**.
2. Click **Sign in with Google** and pick the whitelisted account.
3. You land on your **projects list**. The project the owner shared with you is there — click it to open its board. Once you're inside a project, the back-chevron in the top bar (**Switch project**) jumps between projects.

If you see **"That account is not authorized to use Mykan,"** you signed in with the wrong Google account. Sign out and retry with the whitelisted one.

## 2. (Recommended) Connect your GitHub

If your project uses the GitHub integration (issues/PRs mirrored as cards), connect **your own** GitHub PAT so refreshes and write-backs are attributed to *you*, not a shared identity:

- Click the **GitHub icon** in the top bar and follow the connect flow.
- Full detail — token scopes, org-repo scoping, what syncs — is in `github-integration.md`.

Skip this if your project doesn't touch GitHub.

### How GitHub works on a project

The wiring is set up once per project, usually by the owner:

1. **Project → GitHub account.** Click the **Edit project** pencil next to the project name and pick the account under **GitHub account**.
2. **Area → repo.** Open **Areas** at the right of the board toolbar (on a phone: the filter sheet → **Manage areas**). Hover an area and click **+ repo** to link it to a repo under that account.

After that, these are the controls, and exactly what each one does:

| Control | Where | What it does | What it does **not** do |
|---|---|---|---|
| **Sync** | Right of the board toolbar (on a phone: the filter sheet → **Sync with GitHub**), or *"sync KANBAN with GitHub"* | Imports new open issues from **every** repo-linked area, then moves each card whose issue was **closed on GitHub** to Done. A line under the toolbar lists what moved and what came in | Never reopens a card, never changes content, and doesn't touch the issues |
| **Import** | Areas panel, on a repo-linked area | Creates a Not Started card for each **open** issue in that repo that doesn't already have a card in the project (archived cards count, so they don't come back) | Never changes an existing card, and ignores closed issues |
| **Refresh** | A linked card's GitHub details on its card page (or *"refresh KANBAN-N from GitHub"*) | Pulls that one issue's title, body and labels onto the card, overwriting them (the old version stays in history) | Never changes status, area or assignees |
| **Done → closes the issue** | Automatic, whenever a linked card moves | Moving a card to Done closes its issue; moving it out of Done reopens it, using **your** GitHub token | — |
| **"not synced" badge** | On a card whose close/reopen failed | Click to retry. If you have no GitHub token connected, connect one (above) first | — |

Status flows **from mykan to GitHub** on every move. The one way it flows back is **Sync**: an issue closed on GitHub itself leaves its card where it is until someone runs Sync (or moves the card). Sync only shows once the project is linked to a GitHub account. The board toolbar's **Refresh** only reloads the board from mykan; it doesn't contact GitHub.

## 3. Connect Claude Code over MCP

This is what lets Claude Code (interactive **and** headless/cron) list your projects, move cards, edit items, and drive the GitHub tools **as you**. You need Claude Code installed first.

### a. Mint your personal token

1. In mykan's top bar, click the **key icon** (🔑, next to the GitHub icon) → the **MCP access tokens** panel opens.
2. Optionally type a **label** (e.g. `laptop`, `cron agent`) so you can tell tokens apart later.
3. Click **Generate token**.

4. A `mk_…` value appears with **"Copy this token now — it won't be shown again."** Copy it immediately — only its hash is stored, so it is **unrecoverable** afterward. If you lose it, just generate another and revoke the old one.

The token carries **your identity and your GitHub PAT reach** — treat it like a password. Revoke any token from the same panel (the ✕); revocation is immediate.

### b. Register with Claude Code

The panel shows a ready-to-paste command right under the token. It registers the server at **user scope** — available in every project on your machine, no repo secret, no approval prompt:

```
claude mcp add --transport http --scope user mykan \
  https://kanban.dbwoodward.com/mcp \
  --header "Authorization: Bearer <your mk_… token>"
```

The **same** command and token work for headless/cron agents — there is no browser step. (Interactive browser OAuth — an "Authenticate → Google" button — is **not** built; the token is the intended mechanism and is actually simpler for headless use.)

### c. Verify

```
claude mcp get mykan          # should show ✔ Connected
```

Then in a Claude Code session, `/mcp` (or `/tools`) should list the mykan tools: `mcp__mykan__list_projects`, `mcp__mykan__update_item_status`, and the rest. Your token lives only in `~/.claude.json` on your machine — never in the repo.

> If you registered before new tools shipped, restart the session — MCP tool lists load once per session.

## 4. Work

Ask Claude Code naturally — it maps to the tools:

- *"List my mykan projects / items in Permit Saige"* → `list_projects`, `list_items`
- *"Move KANBAN-N to In Progress / Done"* → `update_item_status`
- *"Create a task in Permit Saige: …"* → `create_item`
- *"Refresh KANBAN-N from GitHub"* → `refresh_item_from_github`
- *"Sync KANBAN with GitHub"* → `sync_project_with_github`
- *"Show me KANBAN-N"* → `get_item` (description, decisions, open questions, progress summary; attached files via `get_attachment`)
- *"Tag / assign / re-parent KANBAN-N"* → `set_item_tags`, `set_item_assignees`, `set_item_parent`, `set_item_area`

### How a card is organised

Every card has more than a description, and Claude Code is told to keep each thing in its place:

- **Description** — a clean, living spec (objective, scope, plan, acceptance), edited in place with `set_item_body` when the plan changes. Old versions are kept in the item's history.
- **Progress notes** — one short checkpoint per meaningful step ( `append_item_note` ). Progress never goes in the description.
- **Questions** — anything the card leaves undecided ( `ask_question` ), closed with `answer_question` once the owner decides.
- **Decisions** — what the project owner decided ( `record_decision` ). Your agent records decisions; it doesn't make them.

Entries are editable and versioned ( `update_item_entry` , `list_item_entry_versions` , `restore_item_entry_version` ), and each is capped at 2,000 characters. Put longer detail in the repo or the PR and link it.

Full tool list and semantics: `mcp-setup.md` .

If the owner installed the **work-item** skill for you, you can also say *"work the next Permit Saige item"* and Claude Code will pick it up, move it to In Progress, do the build from its description, and move it to Done.

---

## Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| "That account is not authorized" on sign-in | Wrong Google account, or not whitelisted yet | Use the whitelisted account; if it still fails, the owner needs to whitelist your email (for Gmail, dots and `+tags` are ignored, so a different spelling of the same address is fine) |
| Signed in, but the project isn't listed | Project not shared with you | Ask the owner to share it (adds you to the project's members) |
| `claude mcp get mykan` shows not connected | Bad/typo'd token, or token revoked | Re-mint a token (§3a) and re-run the `claude mcp add` command |
| Tools missing in a session after connecting, or calls fail after a mykan deploy | Tool list and MCP session are cached for the Claude Code session | Restart the Claude Code session (or reconnect with `/mcp` ) |
| GitHub refresh/write-back fails | No GitHub PAT connected, or wrong scopes | Connect your PAT (§2); see `github-integration.md` for scopes |

| Symptom | Cause | Fix |
| --- | --- | --- |
| An issue was closed on GitHub but its card is still open | Status only comes back from GitHub when someone syncs | Click **Sync** on the board toolbar (§2) |

## See also

- `mcp-setup.md` — the MCP server, tools, and how auth works
- `github-integration.md` — GitHub PAT setup and sync
- `DESIGN.md` — mykan's interaction patterns